

Análise dos Arquivos do Simulador Mancha

Descrição técnica detalhada dos componentes do simulador do processador hipotético Mancha Completo

Este documento apresenta a explicação técnica e funcional de cada um dos arquivos fornecidos para a implementação do **Simulador do Processador Mancha Completo** (baseado na especificação técnica de Marcelo Pasin, 2004). O simulador implementa a arquitetura Von Neumann de 16 bits, incluindo modos Usuário/Supervisor, controle de memória virtual (segmentação e paginação), interrupções de hardware/software, e dispositivos periféricos (console, disco e relógio).

Visão Geral da Arquitetura do Software

Arquivo / Módulo	Papel na Arquitetura	Mecanismos Principais Atendidos
cpu.c	Núcleo de Processamento (CPU)	Ciclo de busca/execução, decodificação, ALU, tradução de memória (MMU/Paginação/Segmentação), ciclo de interrupções.
dispositivos.c	Subsistema de E/S e Periféricos	Mapeamento de portas E/S, controlador de interrupções, console ASCII, disco rígido (DMA sim/ticks) e relógio programável.
instrucao.c	Tabela e Decodificador ISA	Tabela Mnemônica, codificação/decodificação binária de 16 bits (4 formatos), desassemblador (disassembler), modos de endereçamento.
main.c	Ponto de Entrada e Interface	Loop principal de simulação, interface visual Curses, interpretador de comandos interativos do operador (E, Z, D, 1, C, P, R, F).
memoria.c	Memória Física RAM	Alocação do vetor contíguo de bytes (1 MB/64 KB), leitura/escrita de bytes e palavras de 16 bits em formato <i>big-endian</i> .
montador.c	Compilador/Montador ASM	Montador de 2 passos, tabela de símbolos, resolução de pseudo-instruções (.text, .data, .equ, .dw, etc.), geração do código objeto .mob.
objeto.c	Carregador de Objeto (Loader)	Leitura e escrita de arquivos objeto no formato mob j 1, carga de segmentos de memória e resolução de símbolos públicos.
tela.c	Interface Gráfica de Terminal	Renderização ncurses da CPU (registradores r0-r7, s0-s7, flags S/P/I/D/N/O/Z/C), janela de memória traduzida ao redor do IP e saída da console.

[fonte: 1] `cpu.c`

Núcleo do Simulador

Processador, MMU, ALU e Controle de Execução de Instruções

O arquivo `cpu.c` é a peça central do simulador. Ele implementa a estrutura do processador (registradores de usuário `r0-r7` e supervisor `s0-s7`), o ciclo completo de busca, decodificação e execução de instruções, além da Unidade de Gerenciamento de Memória (MMU) que realiza a tradução de endereços virtuais para físicos por **segmentação** (usando CS/CL/DS/DL) ou **paginação** (usando a Tabela de Páginas apontada por PT). Ele também gerencia o vetor e salvamento de contexto de interrupções/exceções na pilha.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `traduz()` : Traduz endereços virtuais de 16 bits para endereços físicos de memória real, verificando violação de segmento ou ausência/permissão de quadro de página.
- `cpu_executa_1()` : Executa exatamente 1 instrução (ou atende 1 interrupção pendente se liberada), controlando incrementos de IP e avanços de ticks nos dispositivos.
- `executa()` : Unidade Lógica e Aritmética (ALU) e controle de desvios. Executa instruções de movimentação (`ld` , `st`), aritmética (`add` , `sub` , `mul` , `div`), lógicas (`and` , `or` , `xor` , `shl` , `shr`) e de controle (`jmp` , `bra` , `call` , `ret` , `trap`).
- `cpu_interrompe()` : Aciona o mecanismo de interrupção/exceção. Salva todo o contexto (16 registradores) na pilha do supervisor e carrega o novo vetor do quadro correspondente.

[fonte: 2] `dispositivos.c`

Hardware Periférico

Controlador de Interrupções e Dispositivos de E/S Mapeados em Portas

Este arquivo simula o hardware dos periféricos conectados ao barramento do Mancha através de portas E/S (acessadas pelas instruções `in/out`). Ele implementa três dispositivos principais: **Console** (buffer circular de entrada/saída de texto), **Disco Rígido Simulado** (leitura/escrita de setores de 512 bytes via DMA com latência simulada por ticks) e **Relógio (Timer)** com contador e limite programáveis para interrupções periódicas. Também contém o PIC (Controlador Interrupções) com máscara configurável.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `disp_le_byte()` / `disp_escreve_byte()` : Realizam a leitura e escrita nas portas registradas (Console: 0x0001-0x0002; Disco: 0x0010-0x0015; Relógio: 0x0020-0x0023; Interrupções: 0x0030).
- `disp_tick()` : Avança os relógios do sistema a cada ciclo de instrução, decrementando o tempo de operações de disco em andamento e incrementando o timer do relógio.
- `dispara_operacao_disco()` / `completa_operacao_disco()` : Gerencia transferência direta de dados (DMA) entre os setores do arquivo de disco e a memória RAM principal.

Tabela de Mnemônicos, Decodificação e Desmontagem de Instruções

O arquivo `instrucao.c` encapsula todo o conhecimento da linguagem de máquina do processador Mancha. Ele contém a tabela master (`TABELA[]`) com todas as instruções válidas, seus formatos binários (Registrador 00, Especial 01, Condicional 10, Rápido 11) e os opcodes correspondentes. É responsável por empacotar dados assembly em instrução binária de 16 bits, decodificar instruções binárias e gerar a representação em texto (disassembler/desmontador) para exibição na interface.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- `instrucao_decodifica()` : Analisa a palavra de 16 bits lida da memória e extrai formato, opcode, registradores, modo de endereçamento, bit de tamanho (byte/word), e constantes imediatas (IM6 / IM10).
- `instrucao_empacota()` : Monta uma palavra binária de 16 bits combinando campos de formato, opcode e registradores/modos.
- `instrucao_desmonta()` : Converte a instrução decodificada de volta para a sintaxe assembly humana (ex: `ld r0, (r2+)`).
- `cond_verdadeira()` : Avalia se uma condição de desvio (Z, EQ, C, LO, N, NZ, GE, LT, etc.) é verdadeira com base nas flags N, O, Z, C.

Ponto de Entrada (Entry Point) e Controlador do Simulador

O arquivo `main.c` conecta todos os módulos. Ele inicializa a memória, os dispositivos, a CPU e carrega o arquivo executável (`.mob`) passado via linha de comando (além de montar imagens de disco informadas com a opção `-D`). Ele contém o loop principal de execução (event loop) que lê os comandos digitados pelo operador via Curses e gerencia os modos de execução: passo a passo (1) ou contínuo (C) controlando a taxa de velocidade da simulação.

COMANDOS DE OPERADOR SUPORTADOS NO LOOP:

- `E<texto>` : Insere uma linha de texto na fila de entrada da console simulada.
- `1` / `C` / `P` : Executa 1 passo, Continua a execução do programa ou Para a simulação.
- `D<n>` : Ajusta o atraso/velocidade de execução (níveis 0 a 9).
- `R` / `Z` / `F` : Reinicia a CPU, Limpa a tela da console ou Finaliza o programa simulador.

Gerenciamento da Memória Física Principal (RAM)

Arquivo responsável por abstrair a memória RAM física do computador simulado. Ele aloca um bloco contíguo de bytes e fornece rotinas seguras para leitura e escrita de bytes isolados e palavras de 16 bits (words). Como o Mancha é uma arquitetura **big-endian**, a gravação/leitura de palavras de 16 bits organiza o byte mais significativo no endereço $\$E\$$ e o menos significativo em $\$E+1\$$.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- **mem_le_byte()** / **mem_escreve_byte()** : Acessam endereços individuais aplicando máscara de limite de memória.
- **mem_le_palavra()** / **mem_escreve_palavra()** : Efetua a combinação ou divisão dos dois bytes da palavra de 16 bits em ordem big-endian.

Montador (Assembler) de Dois Passos para Código Fonte Assembly Mancha

O **montador.c** é um utilitário autônomo responsável por traduzir arquivos de código fonte em linguagem de montagem (.asm) para arquivos executáveis em código objeto (.mob). Ele opera em **dois passos**: no primeiro passo ele calcula os endereços e preenche a Tabela de Símbolos com rótulos e constantes (.equ); no segundo passo ele resolve as expressões, calcula deslocamentos relativos e emite os opcodes binários e pseudo-instruções de dados (.db, .dw, .ds).

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- **parse_data()** : Identifica sintaticamente os 7 modos de endereçamento no assembly (direto, indireto, pós-incremento, pré-decremento, imediato, absoluto e deslocamento).
- **processa_pseudo()** : Processa pseudo-instruções como **.text**, **.data**, **.bss**, **.equ**, **.org**, **.db**, **.dw** e **.pub**.
- **monta_instrucao()** : Valida operandos, calcula valores relativos para saltos (**bra**, **brac**) e gera os bytes binários no arquivo objeto de saída.

Leitor e Gravador do Formato de Arquivo Objeto (mobj 1)

Este arquivo implementa as funções de manipulação e carga do formato de arquivo objeto textual do Mancha (cabecalho iniciado por mobj 1). Ele é utilizado tanto pelo montador para gravar os resultados da montagem quanto pelo simulador no momento do boot para carregar o programa diretamente para a memória RAM simulada.

PRINCIPAIS RESPONSABILIDADES E FUNÇÕES:

- **obj_carrega()** : Lê o arquivo **.mob**, parseia as linhas de código/dados (prefixos **T** e **D**) e grava os bytes na memória física. Também extrai a lista de símbolos públicos (prefixo **P**).
- **obj_escreve_byte()** / **obj_escreve_publico()** : Escreve no arquivo de saída as linhas codificadas em hexadecimal.

Renderizador de Interface de Texto e Painel do Operador

O `tela.c` é responsável por toda a interface gráfica em modo texto (usando a biblioteca `ncurses`). Ele desenha de forma organizada o estado do processador em tempo real, permitindo que o usuário visualize a execução do programa e o estado interno do hardware.

ELEMENTOS RENDERIZADOS NO PAINEL:

- **Caixa de Console:** Exibe o histórico de saída de texto gerado pelos programas.
- **Registradores de Usuário & Supervisor:** Exibe `r0-r4`, `bp`, `sp`, `ip` e `sr`, `s1-s3`, `cs/pt`, `c1`, `ds`, `d1` com destaque em cores para o modo supervisor.
- **Flags do SR:** Exibe o estado das flags de controle (`S`, `P`, `I`, `D`) e de condição (`N`, `O`, `Z`, `C`).
- **Janela de Memória:** Exibe a memória virtual ao redor do registrador `ip`, destacando a palavra da próxima instrução em amarelo.
- **Desmontador em Tempo Real:** Mostra o assembly da próxima instrução prestes a ser executada.